- 项目报告（课程第一阶段）
  - 1. 项目目标与范围
  - 2. 架构与工程设计
  - 3. MongoDB 设计
  - 4. 题型、校验及跳转
  - 5. 提交、统计与安全口径
  - 6. 测试与当前证据
    - 三个工程阶段
  - 7. 问题与解决方案
  - 8. AI 使用说明

项目报告（课程第一阶段）

学号：10245501420　姓名：庞毅飞　仓库链接：https://github.com/pyf060502-glitch/Course-of-Modern-Data-Management-System/tree/main/project1

1. 项目目标与范围

我们需要开发一个基于 MongoDB 的简化在线问卷系统，包含以下功能：注册登录、作者管理和发布、四种题型、数据驱动校验与跳转、登录或匿名填写、独立提交及作者统计。重点是数据库设计、后端逻辑、说明文档和测试，不采用现成问卷框架或关系数据库。

项目开发分为两个阶段：在第一阶段下，我们需要实现系统的可运行基础；第二阶段则要根据尚未公布的新要求扩展，保留核心结构、历史数据与主要功能，不推倒重做。下面是第一阶段的工作内容。

2. 架构与工程设计

根据指导文件提示，我选择了 Node.js + Express 单体应用、MongoDB Node.js Driver、HTML/CSS/原生 JavaScript 作为基本架构；使用路由处理 HTTP、服务处理业务、数据库模块管理连接与索引、独立函数处理校验和跳转；会话部分采用 express-session/connect-mongo，密码部分采用 bcryptjs。最后，在测试部分采用 Node 内置测试运行器与 Supertest，集成测试使用真实 MongoDB。

3. MongoDB 设计

在数据库部分，我们使用了四个集合，分别保存用户、问卷、回答和会话。各集合的职责与关系如下，更详细的字段和索引说明见 数据库设计。

集合	职责与关系
users	保存用户名、规范化后的唯一用户名、bcrypt 哈希和注册时间
surveys	保存作者引用、问卷设置、状态和截止时间；题目及问卷级跳转规则嵌入问卷文档
responses	独立保存每次提交，引用问卷和登录用户，匿名用户引用为 null；同时保存规范化答案、服务端路径和提交时间

集合	职责与关系
sessions	保存由 connect-mongo 管理的服务端会话，使用 expires TTL 管理过期，不作为问卷业务实体

由于题目和规则通常需要整体读写，我选择将它们嵌入问卷文档，这也比较符合 MongoDB 的文档模型。回答数量则会随着提交不断增长，因此使用独立集合保存，避免问卷单个文档受到 16 MiB 上限的限制。对于用户、题目和选项，我们使用引用或稳定 ID，不重复复制用户名、标题和标签；发布后冻结问卷结构，以保证历史答案的含义不变。seed 会在 users/surveys 中添加可选的 seedTag，用来识别固定演示数据；API 不允许提交该标记，公开投影也不会返回它。

在索引部分，我们通过规范化用户名的唯一索引保证账号唯一，通过问卷作者/创建时间以及回答问卷/提交时间、问卷/填写者索引支持列表和统计。对于禁止重复填写的登录提交，使用字符串 singleResponseKey 的 partial unique 索引，其他提交则省略该字段。sessions 的过期由 TTL 管理，但问卷截止不使用 TTL 删除数据。

问卷保存的状态为 draft、published 和 closed。当状态为 published 且 deadline 小于或等于服务器时间时，系统会实时派生 expired 状态，不覆盖已关闭状态。正常删除时，系统先关闭问卷，再删除回答和问卷。不过，在途提交与删除之间不保证跨集合原子性，因此使用时应先关闭问卷，并确认无人填写。

4. 题型、校验及跳转

根据课程要求，我们实现了四种题型，存储时分别使用 single、multiple、text 和 number，对应单选、多选和两类填空题。题目和选项 ID 由服务端生成，修改标签不会改变选项 ID，题目排序从 1 开始并保持连续。题型配置可以包含多选 min/max/exact、文本长度、数字范围和整数限制；对于非法、不一致或不可满足的配置，系统会拒绝保存。

在答案校验部分，单选题需要检查选项是否合法；多选题需要检查答案是否为数组、选项是否合法且无重复，以及选择数量是否符合要求。文本答案经过 trim 后保存，并按 Unicode code point 计数；数字答案必须是 JSON number，其中 0 是有效值，字符串、不安全整数和非有限数都会被拒绝。可选题的空答案不应用非空数量限制，也不匹配跳转规则；实际访问到的必答题则必须填写。

在跳转部分，我们将规则保存在问卷数据中，并在代码中实现允许使用的运算符，数据库不保存可执行代码。单选题支持 equals/not_equals，多选题支持 contains/not_contains/contains_any/contains_all，数字题支持六类比较，文本题在首版中不支持跳转。跳转目标只能是后续题目或 END，同一来源题目的规则优先级必须唯一，较小值优先匹配；没有匹配规则时，系统按题目顺序继续，以避免循环。

对于 normalizeQuestion 请求，text/number 类型不允许包含 options，空数组也不例外，但持久化响应仍统一返回 options: []。新增选项时省略 id，修改已有选项名称时使用本题的选项 id；validation 采用整体替换方式，修改 type 且未传入 validation 时，将其重置为 {}。规则 id 可以由服务端生成，也可以使用合法且在本次数组中唯一的 UUID，不要求该 id 已经存在。管理问卷接口直接在 data 中返回完整持久化字段，并补充 id/effectiveStatus/publicUrl；新增题目返回 data.question，修改题目返回 removedRuleIds，删除问卷返回 deletedResponses。

填写时，前端逐题展示，允许返回修改，并删除不再可达的旧答案。后端则独立从首题重放路径，不信任客户端提供的路径，只校验实际访问的题目，并拒绝路径外、未知或重复的答案。被跳过的必答题无需填写，到达 END 后直接结束问卷。删除题目时会清理关联规则；如果排序导致规则向前跳转，系统会拒绝该排序。

5. 提交、统计与安全口径

在参与设置部分，系统默认要求登录，并允许重复填写，每次填写都会保存为独立记录。只有匿名=true 且重复=true 时，才允许未登录提交；禁止重复填写时，即使允许匿名，也必须登录，因为匿名 Cookie 不能可靠识别填写者。登录提交使用真实会话身份，不接受客户端传入的作者、填写者、匿名标记或提交时间。

提交能否成功，以服务端最终保存前的状态检查为准。如果问卷已关闭或到期，接口返回 410，回答不写入数据库。禁止重复填写依赖 MongoDB 唯一索引处理并发，重复提交返回 409。完成页只显示通用提示，不公开回答内容或填写者身份。

统计部分使用 MongoDB 聚合生成结果，只允许问卷作者访问。其中，totalResponses 表示提交次数，重复提交会重复计数；uniqueRegisteredRespondents 表示去重后的登录用户数，anonymousResponses 表示匿名提交数，不能将其理解为匿名人数。逐题统计区分 visitedCount 和 answeredCount，单选比例的分母为该题非空有效回答数，多选题每条记录中的同一选项最多计数一次。文本和数字内容采用分页，默认 20 条，最大 100 条；数字题没有回答时，count=0，均值与极值为 null，零票选项仍会返回。

在认证部分，注册后不会自动登录，me 返回 user/null；登录时重建会话并返回新的 csrfToken，写请求使用 X-CSRF-Token；退出时销毁会话并清除 Cookie。自动化测试已经验证了这些后端行为。不过，生产环境启用 Secure Cookie 并不代表已经完成 HTTPS 部署；当前程序实际使用 HTTP 监听，且未设置 trust proxy，代理信任与 TLS 需要在部署时另行配置。

6. 测试与当前证据

在测试部分，测试用例保留了原计划中的 15 个用户测试组和 18 个边界测试组，并逐项补充了实际断言、输入差异以及浏览器 V01～V05 的结果。查看结果时，应以实际回填表为准，而不是初稿中保留的“待运行”历史栏。持久化测试使用真实 MongoDB；对于需要控制执行时机的测试，我们包装了真实数据库方法，因此不能说测试中完全没有使用测试替身。

在后续修复中，删除确认修复后的完整测试达到 66/66，发布和关闭确认修复后达到 **88/88**，其中确认操作回归测试为 34 项。这是目前已有的最新完整成功记录，初版 54/54 及其耗时仅作为历史结果保留。

范围	已有测试记录	证据与限制
基础安装与配置	历史记录显示 npm install、npm ci 成功，0 vulnerabilities	依据测试结果中的历史执行记录
基础层自动化	历史最终结果为 8 tests、8 pass、0 fail、0 skipped，7071.5727 ms	仅覆盖基础层，不等同于全系统验收
真实数据库	历史记录已核验 ping、业务索引幂等、唯一索引并发和测试库清理	使用第二连接读取数据，不能代替完整应用重启测试
基础 HTTP 与失败处理	历史记录已核验 health 200/503、异常和配置失败	超限 JSON 返回 413，具体命令见测试结果
认证与业务	阶段反馈包括认证加基础 16/16、逻辑完成时完整测试 30/30、管理单独测试 9/9 与当时完整测试 51/51、公	这些范围互有包含，不能求和，历史结果也不能代替最终结果

范围	已有测试记录	证据与限制
	开统计单独测试 11/11 与当时完整测试 52/52	
初版完整自动化	系统验证阶段执行 npm test，结果为 54/54、0 失败、0 跳过，15306.4727 ms	初版历史结果，包含该阶段最终四项系统测试
修复后完整自动化	最新完整成功记录为 88/88，含确认操作回归测试 34 项	原有后端与 MongoDB 测试继续保留；成功记录及本次复测结果见测试结果
HTTP 进程重启	四个集合的完整文档及 BSON 类型保持一致，原 Cookie 有效，过期后被拒绝	system.test.js 实际停止并重新启动子进程，不是仅重建 app
seed 与安装启动	在随机空库中运行两次，结果为 1 user/1 survey/0 responses，BSON unchanged；独立 fixture 保留无关数据，冲突、密码、三类跳转及 END 测试通过；npm ci 零漏洞，start/dev 的 health 均返回 200	只清理自建 fixture，不删除未知旧库；尚未验证 watch 在修改源码后的热重启
页面资源	七个 HTML 页面以及公共、作者、填写和统计脚本已存在，默认接线 HTTP 测试覆盖页面壳	页面返回 200 或通过静态检查，不等同于交互验收
最终浏览器	使用本机 Chrome 和真实 DOM，窗口为 1280×900，主流程 9/9、补充 4/4，pageerror=0；V01～V05 实际步骤通过	证据包括 TEMP 结果 JSON 和统计截图的视觉复核；脚本不属于默认 npm test，具体路径见测试结果

实际自动化测试与原计划存在一些输入差异：统计测试实际使用 4 次提交、2 个登录用户和 1 次匿名提交，数字为 2 和 8，平均值为 5，并不是计划中的 5 次提交、2 次匿名提交以及数字 0、10、20；数字比较单元测试的阈值为 2，不是 18；禁止重复填写的集成测试实际使用 6 个并发请求，结果为 1 个成功、5 个冲突。因此，不能因为已有完整测试达到 88/88，就认为所有尚未逐字执行的容量和具体输入组合都已经测试过。

三个工程阶段

为了说明开发过程，下面按照实现能力，将实际文件和阶段测试结果整理为三个工程阶段。表中保留开发时的路径简称；搬迁后程序文件统一位于 `code/`，说明文档位于 `文档/`，测试文档位于 `测试/`，本报告和 README 位于项目根目录。`.gitignore` 仍位于项目根目录，npm 命令在 `code/` 中执行。

阶段	新增/修改文件与实现能力	实际命令与结果
一：基础与管理	包含 package.json/lock、.env.example、.gitignore；src/config.js、app.js、server.js、db/、*middleware/*、routes/auth.js/surveys.js、services/surveys.js/jump.js、validators/*；foundation/auth/logic/surveys 测试及数据库设计。该阶段实现配置、真实存储、认证、四种题型、规则、作者 CRUD 和问卷生命周期	npm install/npm ci 均为 0 漏洞；首次 npm test 因 Windows 通配符问题失败，改用 node --test tests 后基础测试达到 8/8；随后记录 16/16、30/30、管理单独测试 9/9 及当时完整测试 51/51 均通过

阶段	新增/修改文件与实现能力	实际命令与结果
二：填写与统计	包含 src/routes/public.js/statistics.js、services/responses.js/statistics.js、app 接线和 tests/responses-statistics.test.js。该阶段实现公开投影、参与限制、独立回答、唯一索引并发处理、最终状态检查、路径重放、聚合和分页	公开统计单独测试达到 11/11，当时完整 npm test 达到 52/52；6 个并发请求中，1 个返回 201，5 个返回 409，统计与分页的实际断言详见测试用例
三：页面与交付	包含 public 的七个页面及 css/js；scripts/seed.js、tests/system.test.js；README 和文档、测试目录中的交付文档。该阶段实现账号和作者表单、逐题返回、统计内容页、幂等示例及文档；后续增加 tests/deletion-ui.test.js，修复删除、发布和关闭确认操作	安装启动验证记录为 npm ci 零漏洞，npm start/dev 的 health 返回 200，seed 两次运行的 BSON 一致；初版 npm test 为 54/54，15306.4727 ms，修复后完整成功记录为 88/88；浏览器主流程 9/9、补充 4/4，确认修复另有真实 Chrome 验证记录

7. 问题与解决方案

在开发和验证过程中，我们遇到了以下问题，并根据实际情况进行了处理。对于尚未解决或尚未验证的部分，也保留了明确说明。

问题	解决方案与实际处理	当前限制
Windows Node 20 不展开 tests/*.test.js	历史 AI 实施中改用 node --test tests，随后真实基础测试达到 8/8	此处解决的是基础测试发现的问题，不能据此得出全系统测试结论
工程过度复杂与阶段安排	根据需求反馈，采用三个小阶段和简单模块架构	第二阶段继续保留核心结构并扩展功能
匿名者不能可靠防重复	禁止重复填写时要求登录，并通过 partial unique 索引保障并发	匿名提交数不能理解为匿名人数
跳转造成隐藏答案或漏填	前端清除不可达答案，后端重放路径	后端自动化以及浏览器 Full→Skip 清除 Q2、END 的实际步骤已通过；原计划中的反向路径没有逐字执行，不能写成已执行
历史答案含义可能变化	使用稳定 ID，并在发布后冻结结构	新题型或变更需要按第二阶段兼容性方案追加
删除与在途提交竞争	先关闭问卷，确认无人填写后进行正常级联删除	不保证跨集合原子性，也没有实现补偿机制

问题	解决方案与实际处理	当前限制
交付草稿与管理实现不一致	根据实际 src 修正了 data 结构、分页、选项 ID、type/validation 及关闭限制	文档核对和最终浏览器证据回填已完成，同时保留历史范围和未执行部分的说明

8. AI 使用说明

在 AI 使用部分，我主要负责提出需求、反馈工程方案、检查使用效果并提出调整要求；AI 负责辅助梳理业务、生成或修改代码、补充测试和整理文档。提出修改要求与亲自编写代码是不同的工作，记录中分别说明。

根据实际开发过程，AI 的帮助、出现的问题和我负责的部分如下：

- AI帮了什么:帮助梳理系统架构，生成MongoDB数据操作、后端接口、校验与跳转逻辑，并补充测试和说明文档。
- AI做错了什么:最初的删除、发布和关闭确认逻辑存在问题，取消后仍可能执行操作;Windows 下的测试命令也出现过通配符问题，后续均进行了修复。
- 自己改了什么:我调整了交付目录，将程序文件集中到code/中，并检查和修改文档内容;界面提示及确认逻辑由我提出修改要求，再由AI实施代码调整。
- 自己设计了什么:我根据课程要求确定功能范围、简化架构和分阶段开发安排，并组织代码、文档及测试的交付结构;具体数据库和后端实现由AI辅助完成。

具体工具、提示词、生成结果及修改情况见 AI 使用日志。对于没有完整保留的历史提示，只记录能够确认的摘要；概念学习、实际代码修改和测试验证也分别说明。